

Advanced techniques in applied economics

Lecture 5: Latent variable models — from trees to neural nets

Piotr Zwiernik



**Universitat
Pompeu Fabra**
Barcelona

Statistics, Probability
and Machine Learning
Research Group



Barcelona School of Economics

Spring 2026

Latent structure as hidden economic mechanism

What changes today?

In the previous lectures, graphs described dependence among *observed* variables. Today we allow some important variables to be **hidden**.

Why hidden variables?

- market-wide shocks are not directly observed,
- state of the economy is only visible through some indicators
- consumers or firms belong to hidden segments,

Dimension reduction: PCA and related methods assume latent structure in the data.

Main question: Can a small number of latent variables explain complicated patterns in the data?

Main message: latent variables can turn a complicated observed dependence pattern into a simpler hidden structure.

Roadmap for today

1. factor analysis and hidden common factors
2. Gaussian mixtures and latent classes
3. latent tree models and hierarchical structure
4. restricted Boltzmann machines as a bridge to neural nets

When dependence is too complicated at the surface, look for a simpler hidden layer underneath.

Part 1: Factor analysis

Why economists care about factor models

Finance

A large collection of returns often moves because of a few common factors:

- market return,
- sectoral shocks,
- liquidity or volatility factors.

Applied micro and macro

Latent variables can represent

- ability,
- sentiment,
- business-cycle conditions,
- hidden household or firm types.

Factor model: many variables may comove because they are driven by the same hidden source.

For standard treatment, see Chapter 9 in Hastie, Tibshirani, and Friedman, *The Elements of Statistical Learning*.

The factor analysis model

The basic Gaussian factor model is

$$X = \mu + WZ + \varepsilon, \quad Z \sim N_r(0, I_r), \quad \varepsilon \sim N_m(0, \Psi), \quad Z \perp\!\!\!\perp \varepsilon,$$

where Ψ is diagonal.

Interpretation

- $Z \in \mathbb{R}^r$ are the latent factors,
- $W \in \mathbb{R}^{m \times r}$ is the loading matrix,
- Ψ captures variable-specific noise not explained by the common factors.

Covariance form

$$\Sigma = WW^T + \Psi.$$

So dependence is split into a **low-rank** common part and a **diagonal** idiosyncratic part.

A one-factor economic example

Suppose m observed returns satisfy

$$X_i = \lambda_i F + \varepsilon_i, \quad i = 1, \dots, m,$$

where F is a latent factor, and all ε_i are independent and independent of F .

Covariance implication

For $i \neq j$,

$$\text{cov}(X_i, X_j) = \lambda_i \lambda_j \text{var}(F).$$

Also,

$$\text{cov}(X_i, F) = \lambda_i \text{var}(F),$$

so

$$\text{cov}(X_i, X_j) = \frac{\text{cov}(X_i, F) \text{cov}(X_j, F)}{\text{var}(F)}.$$

Economic reading

The assets comove because they all load on the same hidden market condition F .

Let $\rho_i = \text{cor}(X_i, F)$ and $\rho_{ij} = \text{cor}(X_i, X_j)$. Then

$$\rho_{ij} = \rho_i \rho_j \quad \text{for all } i \neq j.$$

A simple correlation consequence

One-factor correlation pattern

The observed correlation matrix must have the form

$$\rho_{ij} = \rho_i \rho_j \quad \text{for all } i \neq j.$$

Testable implications

If $1 \leq i < j < k \leq m$, then $\rho_{ij}\rho_{ik}\rho_{jk} = \rho_i^2 \rho_j^2 \rho_k^2 \geq 0$.

So under a one-factor model, the product of three pairwise correlations cannot be negative.

For any $1 \leq i < j < k < l \leq m$, we also have the **tetrad constraints**

$$\rho_{ij}\rho_{kl} = \rho_{ik}\rho_{jl} = \rho_{il}\rho_{jk}.$$

For classical testable implications of latent-variable models, see Spirtes, Glymour, and Scheines, *Causation, Prediction, and Search*, and Drton, Sturmfels, and Sullivant, *Lectures on Algebraic Statistics*.

A tiny one-factor check in R

```
R <- matrix(c(1, 0.6, 0.3,  
              0.6, 1, 0.5,  
              0.3, 0.5, 1), 3, 3)
```

```
rho12 <- R[1,2]
```

```
rho13 <- R[1,3]
```

```
rho23 <- R[2,3]
```

```
rho12 * rho13 * rho23
```

```
# implied squared correlations
```

```
rho2 <- c(rho12*rho13/rho23,  
          rho12*rho23/rho13,  
          rho13*rho23/rho12)
```

```
rho2
```

```
# all should lie in [0,1]
```

```
all(rho2 >= 0)
```

```
all(rho2 <= 1)
```

Interpretation

A negative product $\rho_{12}\rho_{13}\rho_{23}$ rules out a one-factor explanation. If the implied values also lie in $[0, 1]$, then a one-factor explanation is at least algebraically possible.

How many factors should we keep?

This is a real modelling choice: too few factors miss structure, too many overfit noise.

Why raw eigenvalues are not enough? Even if the data contain *no* factor structure, the sample correlation matrix will still have some large eigenvalues just because of sampling variability. So we need a reference point: which eigenvalues are larger than what noise alone would typically produce?

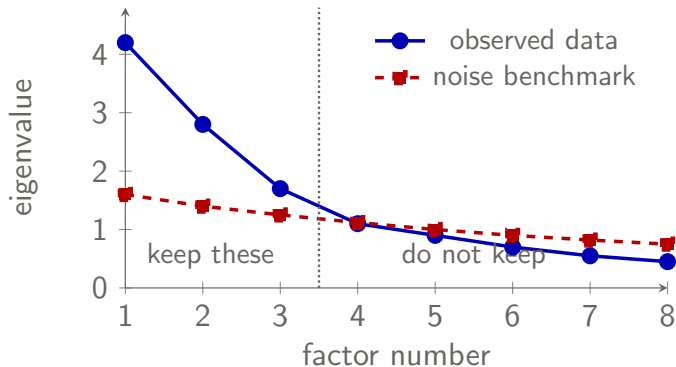
Parallel analysis

Generate random data with the same sample size and dimension, but with no latent factor structure. Then compare the observed eigenvalues to the eigenvalues from this noise benchmark. Keep only those factors whose observed eigenvalues are clearly larger than the benchmark.

Parallel analysis helps separate genuine latent structure from eigenvalues that are large only because of finite-sample noise.

For parallel analysis, see Horn (1965), *Psychometrika*.

Parallel analysis: visual intuition



How to read the plot

Keep factors only while the observed eigenvalues stay above the noise benchmark. In this example, the first three factors stand out from noise, but the fourth does not.

The key comparison is not whether an eigenvalue is large by itself, but whether it is larger than what we would expect under no factor structure.

Choosing the number of factors in R

```
set.seed(1)
library(MASS)
library(psych)

Lambda <- matrix(c(0.8, 0.2,
                  0.7, 0.1,
                  0.6, 0.3,
                  0.1, 0.7,
                  0.2, 0.8), 5, 2,
                byrow = TRUE)
Psi <- diag(c(0.3,0.4,0.5,0.3,0.4))
Sigma <- Lambda %*% t(Lambda) + Psi
X <- mvrnorm(500, rep(0,5), Sigma)

fa.parallel(X, fa = "fa")
```

Interpretation

The plot compares the observed eigenvalues to a noise benchmark and suggests how many factors are worth retaining.

Why rotation matters

Recall: $X = \mu + WZ + \varepsilon$, $Z \sim N_r(0, I_r)$, $\varepsilon \sim N_m(0, \Psi)$, $Z \perp\!\!\!\perp \varepsilon$.

The factor model is not uniquely parameterized. If U is an orthogonal matrix, then

$$WZ = (WU)(U^\top Z),$$

so the same covariance structure can be represented by many different loading matrices.

Why this matters

The fitted model may be correct, but the factors are not automatically easy to interpret.

Varimax

Varimax is an orthogonal rotation that tries to make the loadings simpler: each variable should load strongly on only a few factors, and weakly on the others.

For rotation methods, see Kaiser (1958), *Psychometrika*.

Fitting and rotating a factor model in R

No rotation

```
fit0 <- factanal(X, factors = 2,  
                 rotation = "none")  
fit0$loadings
```

Varimax rotation

```
fit1 <- factanal(X, factors = 2,  
                 rotation = "varimax")  
fit1$loadings
```

Interpretation

Varimax keeps the same fitted covariance structure but often makes the loading pattern easier to interpret.

Application: personality-trait data

We analyze the `bfi` dataset from the `psych` package.

Dataset

- about 2,800 responses,
- 25 personality-related questions,
- items designed to reflect the Big Five personality traits: Neuroticism, Extraversion, Conscientiousness, Agreeableness, and Openness.

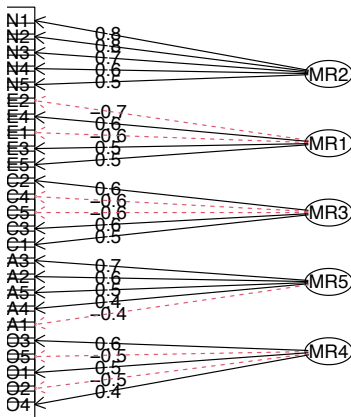
Responses are on a 1–6 agreement scale. We discard demographic variables such as gender, age, and education.

For the `psych` package and the `bfi` data, see Revelle, *psych: Procedures for Psychological, Psychometric, and Personality Research*.

Factor analysis results

Empirical message

Parallel analysis suggests retaining **five** factors, matching the Big Five structure rather well. Varimax-rotated loadings then help align the factors with interpretable trait groups.



Appendix: What is the latent factor given the data?

In the Gaussian factor model

$$X = \mu + WZ + \varepsilon, \quad Z \sim N_r(0, I_r), \quad \varepsilon \sim N_m(0, \Psi),$$

the conditional distribution of the latent factor given the data is Gaussian:

$$Z \mid X = x \sim N_r(m(x), V),$$

where

$$V = (I_r + W^\top \Psi^{-1} W)^{-1}, \quad m(x) = V W^\top \Psi^{-1} (x - \mu).$$

Interpretation

The posterior mean $m(x) = \mathbb{E}[Z \mid X = x]$ is the best estimate of the hidden factor given the observed variables.

This is the Gaussian analogue of extracting the latent factor from the observed data.

These formulas follow from the standard conditional formulas for multivariate Gaussians. See, for example, Bishop, *Pattern Recognition and Machine Learning*, Chapter 12.

Appendix: EM for factor analysis

For each observation x_i , the E-step computes

$$m_i = \mathbb{E}[Z_i \mid X_i = x_i], \quad V_i = \text{var}(Z_i \mid X_i = x_i).$$

E-step

Use the Gaussian conditional formulas to estimate the hidden factors from the current parameters.

M-step

Update μ , W , and Ψ using the estimated first and second moments of the latent factors.

Interpretation

EM alternates between estimating the hidden factors given the current model and updating the model given the current estimated factors.

For EM in latent-variable models, see Dempster, Laird, and Rubin (1977), *JRSS-B*.

Part 3: Latent tree models

Why latent tree models?

Sometimes one global factor is too crude, but a fully general latent structure is too flexible.

Latent tree idea

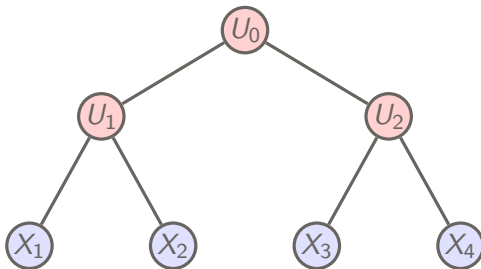
Observed variables sit at the leaves of a tree, and hidden variables occupy internal nodes. Nearby leaves share more hidden ancestry than distant leaves.

Economic reading

This is natural for hierarchical structure: firms inside sectors, sectors inside the macroeconomy, or regional demand inside a national cycle.

Latent tree models sit between one-factor models and completely unrestricted latent networks.

A simple latent tree picture



Interpretation

X_1 and X_2 are similar because they share a local hidden parent U_1 . At a higher level, the two groups are connected through the more global hidden variable U_0 .

Latent tree models as marginals of tree models

Start with a probabilistic model on a tree whose nodes include both observed and hidden variables. A **latent tree model** is what we get after marginalizing out the hidden nodes.

Special cases This includes familiar examples such as

- latent class / naive Bayes models,
- hidden Markov models,
- hierarchical Gaussian models.

Main point

Even if the hidden variables are not observed, the tree still imposes strong structure on the observed joint distribution.

Gaussian latent tree models

Now suppose all variables in the tree are jointly Gaussian and centered.

Edge correlations. For each edge (u, v) in the tree, write

$$\rho_{uv} = \text{cor}(X_u, X_v).$$

Path-product formula

For any two nodes i and j in a Gaussian tree,

$$\text{cor}(X_i, X_j) = \prod_{(u,v) \in \overline{ij}} \rho_{uv},$$

where $\overline{ij}$ is the unique path from i to j .

For Gaussian latent trees, see Zwiernik, *Latent tree models*, in the *Handbook of Graphical Models*.

From correlations to tree metrics

Because correlations multiply along paths, logarithms turn them into sums:

$$-\log |\operatorname{cor}(X_i, X_j)| = \sum_{(u,v) \in \overrightarrow{ij}} -\log |\rho_{uv}| > 0.$$

Tree metric

This means the quantities

$$d(i, j) := -\log |\operatorname{cor}(X_i, X_j)|$$

behave like path lengths on a tree.

This is why pairwise correlations can be used to reconstruct a hidden tree.

A concrete Gaussian latent tree example

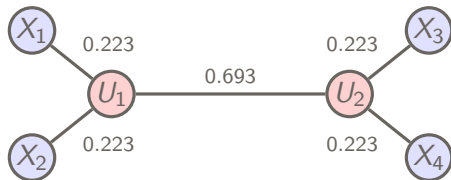
Observed correlation matrix

$$R = \begin{pmatrix} 1 & 0.64 & 0.32 & 0.32 \\ 0.64 & 1 & 0.32 & 0.32 \\ 0.32 & 0.32 & 1 & 0.64 \\ 0.32 & 0.32 & 0.64 & 1 \end{pmatrix}$$

This matrix comes from a latent tree in which the four outer edges have correlation 0.8 and the middle edge has correlation 0.5. For example,

$$\text{cor}(X_1, X_2) = 0.8^2 = 0.64,$$

$$\text{cor}(X_1, X_3) = 0.8 \cdot 0.5 \cdot 0.8 = 0.32.$$



The edge labels are the tree distances

$$-\log(0.8) \approx 0.223, \quad -\log(0.5) \approx 0.693.$$

Applications beyond economics

Latent trees are useful when hidden structure is *hierarchical* rather than global.

Psychometrics

Broad latent traits may split into narrower dimensions, with observed items at the leaves.

Hierarchical topic detection

Observed binary word variables sit at the leaves, while hidden nodes capture word co-occurrence patterns at different levels of specificity.

Phylogenetics

Observed species sit at the leaves, hidden ancestral species lie at internal nodes, and genetic distances behave approximately like path lengths on a tree.

All three examples share the same mathematical pattern: observed leaves, hidden internal nodes, and additive path structure.

Pointers: Reise (2012), *Multivariate Behavioral Research*; Liu, Zhang, and Chen (2014), *Machine Learning*; Semple and Steel, *Phylogenetics*.

Relation to hierarchical clustering

Once correlations are transformed into distances,

$$d(i, j) = -\log |\text{cor}(X_i, X_j)|,$$

the problem starts to look like hierarchical clustering.

But there is a difference

Hierarchical clustering always returns a dendrogram, even if the distances do not come from a tree. Latent-tree reconstruction asks a stronger question: do these distances behave like sums along tree paths?

So hierarchical clustering is a useful exploratory tool, while latent-tree models add a probabilistic interpretation.

A factor model gives a few global hidden forces. A latent tree gives a hidden hierarchy.

A stock-market example: exploratory hierarchy from correlations

The dataset `stockdata` contains stock prices together with sector information. We compute log-returns, transform correlations into distances, and then apply single-linkage clustering.

```
library(huge)
data(stockdata)
set.seed(1)
ns <- 80
sample.stocks <- sample(1:ncol(stockdata$data), ns)

dat <- log(
  stockdata$data[2:1258, sample.stocks] /
  stockdata$data[1:1257, sample.stocks]
)
dat <- scale(dat)

R <- cor(dat)
D <- -log(abs(R))
hc <- hclust(as.dist(D), method = "average")
```

Interpretation

Stocks with strong correlations have smaller distances $d(i, j) = -\log |R_{ij}|$. If the market has hidden hierarchical structure, then firms from the same sector should tend to appear close together.

This is an exploratory construction, not a full latent-tree fit. But it gives a first data-driven picture of possible hidden hierarchy.

Visualizing the stock hierarchy

We can display the result as a dendrogram.

```
labs <- stockdata$info[sample.stocks, 1]

plot(hc, labels = labs,
     cex = 0.55,
     main = "")

# optional: color labels by sector
sector <- as.factor(stockdata$info[sample.stocks, 2])
lab.col <- as.numeric(sector)

plot(hc, labels = FALSE, cex = 0.55, main = "")
text(x = 1:length(hc$order), y = par("usr")[3] - 0.02,
     labels = labs[hc$order], srt = 90, adj = 1,
     xpd = TRUE, col = lab.col[hc$order], cex = 0.55)
```

What to look for

If the construction is meaningful, one may see

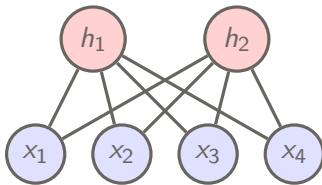
- firms from the same sector merging early,
- related sectors joining at intermediate heights.

Coloring labels by sector helps check whether the recovered hierarchy lines up with economic structure already known from the data.

Part 4: Restricted Boltzmann machines

Restricted Boltzmann machines

A restricted Boltzmann machine (RBM) is a **bipartite undirected latent-variable model** with visible units x and hidden units h .



Restriction

There are no visible-visible edges and no hidden-hidden edges. So the model is simple *conditionally*, even though the marginal dependence structure can be complicated.

For RBMs, see Hinton (2002), *Neural Computation*, and Fischer and Igel (2012), *Pattern Recognition*.

RBM versus factor models

RBM and factor models are both latent-variable models, but their simplifications occur in different places.

Factor model

Usually written as a directed model:

$$X = \mu + WZ + \varepsilon.$$

The latent factors Z are simple *marginally*, often independent Gaussian variables.

RBM

An undirected bipartite model with joint law

$$P(x, h) \propto \exp(a^\top x + b^\top h + x^\top W h).$$

The hidden units are simple *conditionally*: they become independent once x is fixed.

The key simplification in an RBM is not marginal independence of the hidden layer. It is the factorization of the conditional laws $P(h \mid x)$ and $P(x \mid h)$.

Why $P(h \mid x)$ factorizes

Fix x . Then

$$P(h \mid x) \propto \exp\left(b^\top h + x^\top W h\right).$$

Now write

$$b^\top h + x^\top W h = \sum_{j=1}^q h_j (b_j + w_j^\top x),$$

where w_j is the j th column of W . Hence

$$P(h \mid x) \propto \prod_{j=1}^q \exp\left(h_j (b_j + w_j^\top x)\right).$$

Conditional on x , the hidden units are independent

$$P(h \mid x) = \prod_{j=1}^q P(h_j \mid x), \quad P(x \mid h) = \prod_{i=1}^m P(x_i \mid h).$$

From factorization to sigmoid activation

Since each hidden unit is binary,

$$P(h_j = 1 \mid x) = \frac{\exp(b_j + w_j^\top x)}{1 + \exp(b_j + w_j^\top x)} = \sigma(b_j + w_j^\top x),$$

where

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

Likewise,

$$P(x_i = 1 \mid h) = \sigma(a_i + \tilde{w}_i^\top h),$$

where $\tilde{w}_i^\top$ is the i th row of W .

Bridge to neural nets

An RBM hidden unit activates by applying a sigmoid to a linear score. This is one reason RBMs formed a bridge from latent-variable models to neural-network architectures.

What is the learned feature?

For an observation x , the hidden layer defines the vector

$$\phi(x) = (P(h_1 = 1 \mid x), \dots, P(h_q = 1 \mid x)).$$

Factor model

A feature is typically a posterior mean such as

$$\mathbb{E}[Z \mid X = x],$$

which is a Gaussian latent score.

RBM

A feature is the vector of hidden activation probabilities

$$P(h_j = 1 \mid x).$$

This gives an observation-specific nonlinear representation of the input.

So RBMs matter here because they show how a simple latent-variable model can be used not only to explain dependence, but also to **extract features**.

A tiny RBM-style code illustration

In modern practice, one usually works with autoencoders or neural nets rather than classical RBM training, but the basic hidden-layer idea is similar.

```
set.seed(1)
X <- matrix(rbinom(500*4, 1, 0.5),
            500, 4)
W <- matrix(c(1.2,-0.8,
              0.5, 1.1,
              -0.7, 0.9,
              1.0, 0.6), 4, 2,
            byrow = TRUE)
b <- c(0.3, -0.2)
```

```
eta <- sweep(X %*% W, 2, b, "+")
Hprob <- 1/(1 + exp(-eta))
head(Hprob)
```

Interpretation

Each row of Hprob is the feature vector

$$\phi(x) = (P(h_1 = 1 \mid x), P(h_2 = 1 \mid x)),$$

that the RBM extracts from the observed input x .

Take-away

- factor analysis explains dependence through a few hidden common shocks,
- Gaussian mixtures explain dependence through hidden subpopulations,
- latent tree models explain dependence through hierarchical hidden structure,
- RBMs show how latent-variable ideas connect naturally to neural-network representations.

Different latent models explain dependence in different ways: common factors, hidden classes, hierarchies, or distributed features.

Looking ahead

Next lecture

We will return to hidden variables from a more causal perspective: unobserved confounding, proxy controls, and what survives when the clean DAG picture breaks.

Bridge forward

Today the latent variables explained dependence. Next time they will also threaten identification.